Johns Hopkins University

Masters in Artificial Intelligence

Module 9 – CUDA Advanced Libraries

Herbert Schmidmeier

Date: 3/20/2026

Program Description

The program implements two CUDA kernels to convert a BMP image to grayscale and to convert from grayscale to a blurred image. Two testing images are randomly generated to avoid importing a file. The testing images are 7,680 x 4,320. The program will print 5 pixels of each of the testing images at each step of conversion.

```
C:\Users\Herbert\workspace\EN605.617\module9\Assignment_9c>nvcc assignment.cu -o assignment
.exe -lnppc -lnppif -lnppicc -lcurand
assignment.cu
tmpxft_00006f34_00000000-7_assignment.cudafe1.cpp

C:\Users\Herbert\workspace\EN605.617\module9\Assignment_9c>.\assignment.exe

Image size: 7680 x 4320

Test Image 1 - Sample RGB values (RGB):
RED pixels:   21 37 247 246 103
GREEN pixels: 94 90 209 230 255
BLUE pixels:  61 96 248 140 121
Gray pixels: 68 75 225 225 194
Blurred pixels: 67 117 154 185 152

Test Image 2 - Sample RGB values (RGB):
RED pixels:   245 44 173 229 229
GREEN pixels: 43 31 33 43 63
BLUE pixels:  32 13 168 200 229
Gray pixels: 102 33 90 117 132
Blurred pixels: 104 87 76 92 92

C:\Users\Herbert\workspace\EN605.617\module9\Assignment_9c>
```

Figure 1: Compilation and Execution

Item 1 - Advanced Library 1

Random Image Creation cuRAND

```cpp
// Generate a random RGB image directly on the device using cuRAND.
// ---------------------------------------------------------------------
void create_image_device(unsigned char* d_rgb, size_t rgb_bytes, curandGenerator_t gen)
{
    // Round up to the nearest multiple of 4 (curandGenerate requirement)
    size_t image_byte_count = (rgb_bytes + 3) / 4;

    unsigned int* d_tmp;
    CUDA_CHECK(cudaMalloc((void**)&d_tmp, image_byte_count * sizeof(unsigned int)));

    CURAND_CHECK(curandGenerate(gen, d_tmp, image_byte_count));

    // Reinterpret the random uint bytes directly as unsigned char RGB data
    CUDA_CHECK(cudaMemcpy(d_rgb, d_tmp, rgb_bytes, cudaMemcpyDeviceToDevice));

    CUDA_CHECK(cudaFree(d_tmp));
}
```

Item 2 - Advanced Library 2

Convert RGB to Grayscale with nppiRGBToGray_8u_C3C1R_Ctx

```cpp
// Function converts RGB image to grayscale using the NPP library
// ---------------------------------------------------------------------
void rgb_to_gray_npp(unsigned char* d_rgb1, unsigned char* d_rgb2,
    unsigned char* d_gray1, unsigned char* d_gray2,
    int rgb_step, int gray_step, NppiSize roi,
    NppStreamContext nppCtx1, NppStreamContext nppCtx2,
    cudaEvent_t event1, cudaEvent_t event2)
{
    NPP_CHECK(nppiRGBToGray_8u_C3C1R_Ctx(
        d_rgb1, rgb_step, d_gray1, gray_step, roi, nppCtx1));

    NPP_CHECK(nppiRGBToGray_8u_C3C1R_Ctx(
        d_rgb2, rgb_step, d_gray2, gray_step, roi, nppCtx2));

    // Synchronize streams to ensure grayscale conversion is done before blurring
    CUDA_CHECK(cudaEventRecord(event1, nppCtx1.hStream));
    CUDA_CHECK(cudaEventRecord(event2, nppCtx2.hStream));
    CUDA_CHECK(cudaStreamWaitEvent(nppCtx1.hStream, event1, 0));
    CUDA_CHECK(cudaStreamWaitEvent(nppCtx2.hStream, event2, 0));
}
```

Blur grayscale image with nppiFilterBoxBorder_8u_C1R_Ctx

```
// Function blurs grayscale image using the NPP library
// ---------------------------------------------------------------------
void blur_gray_npp(unsigned char* d_gray1, unsigned char* d_gray2,
    unsigned char* d_blur1, unsigned char* d_blur2,
    unsigned char* h_gray1, unsigned char* h_gray2,
    unsigned char* h_blur1, unsigned char* h_blur2,
    int gray_step, NppiSize roi,
    NppStreamContext nppCtx1, NppStreamContext nppCtx2,
    size_t gray_bytes, size_t blur_bytes)
{
    NppiSize  mask = { 3, 3 };
    NppiPoint anchor = { 1, 1 };

    // Blur grayscale images using box filter by averaging 3x3 neighborhood
    NPP_CHECK(nppiFilterBoxBorder_8u_C1R_Ctx(
        d_gray1, gray_step, roi, { 0, 0 },
        d_blur1, gray_step, roi,
        mask, anchor, NPP_BORDER_REPLICATE, nppCtx1));

    NPP_CHECK(nppiFilterBoxBorder_8u_C1R_Ctx(
        d_gray2, gray_step, roi, { 0, 0 },
        d_blur2, gray_step, roi,
        mask, anchor, NPP_BORDER_REPLICATE, nppCtx2));

    // Copy results back to host
    CUDA_CHECK(cudaMemcpyAsync(h_gray1, d_gray1, gray_bytes, cudaMemcpyDeviceToHost, nppCtx1.hStream));
    CUDA_CHECK(cudaMemcpyAsync(h_gray2, d_gray2, gray_bytes, cudaMemcpyDeviceToHost, nppCtx2.hStream));
    CUDA_CHECK(cudaMemcpyAsync(h_blur1, d_blur1, blur_bytes, cudaMemcpyDeviceToHost, nppCtx1.hStream));
    CUDA_CHECK(cudaMemcpyAsync(h_blur2, d_blur2, blur_bytes, cudaMemcpyDeviceToHost, nppCtx2.hStream));
    CUDA_CHECK(cudaStreamSynchronize(nppCtx1.hStream));
    CUDA_CHECK(cudaStreamSynchronize(nppCtx2.hStream));
}
```

Item 3 - Integration of 2 Advanced Libraries into Single Application

The integration of the libraries is a result of the sequence of generating random images with cuRAND, converting RGB to grayscale with NPP, and blurring the grayscale using NPP.